SOFTWARE ENGINEERING PROJECT REPORT

HOTEL MANAGEMENT SYSTEM (HMS)

Requirements Specification, UML Architectural Design & Prototype Traceability

Standard: IEEE-830 SRS Standard	**Implementation:** Python 3 / Tkinter GUI	**Total Marks:** 20 Marks

TASK 1: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1. System Overview & Intended User Roles

The Hotel Management System (HMS) is a centralized desktop platform engineered to automate hotel operations, guest registration, room inventory tracking, billing calculation, and real-time record searches. The software operates under two primary user roles:

- **Hotel Staff:** Handles operational workflows including guest check-ins, multi-night room bookings, check-outs, dynamic guest searches, and financial receipt issuance.
- **Hotel Administrator:** Retains full operational privileges with added administrative security rights to manage system user credentials (add/delete staff) and manipulate inventory (add/modify room rates and details).

2. Functional Requirements (10 Identifiers)

REQ ID	FUNCTION NAME	DETAILED FUNCTIONAL DESCRIPTION
FR-001	User Authentication	The system shall authenticate users via username/password against `staff.txt`, assigning Operational Access based on Administrator or Staff roles.
FR-002	Staff Account Mgmt	The system shall allow administrators to create new staff/admin accounts and delete existing accounts (preventing self-deletion during active sessions).
FR-003	Room Inventory Creation	The system shall enable administrators to add new room entries with a unique Room Number, Nightly Price ($), Occupancy Capacity, and Room Type.
FR-004	Room Attribute Editing	The system shall permit administrators to update the nightly pricing rate and max occupancy capacity of existing room records.
FR-005	Room Removal	The system shall allow administrators to delete unreserved (vacant) rooms from the active system database.
FR-006	Room Reservation	The system shall allow staff to book vacant rooms by capturing validated guest contact details (Name, Phone, Email) and duration of stay (Nights).
FR-007	Guest Departure	The system shall process guest check-outs by transitioning room status from 'Occupied' back to 'Available' and clearing assigned guest fields.
FR-008	Receipt Generation	The system shall automatically generate itemized financial receipts detailing booking metrics and export printable text files (`Receipt_Room_X.txt`).
FR-009	Dynamic Search	The system shall dynamically filter active room listings upon key release in real-time across parameters: Room #, Guest Name, Phone, Email, Type, or Capacity.
FR-010	Data Persistence	The system shall maintain persistence by auto-syncing all room and account states to local data files (`data.txt` and `staff.txt`) upon every state change.

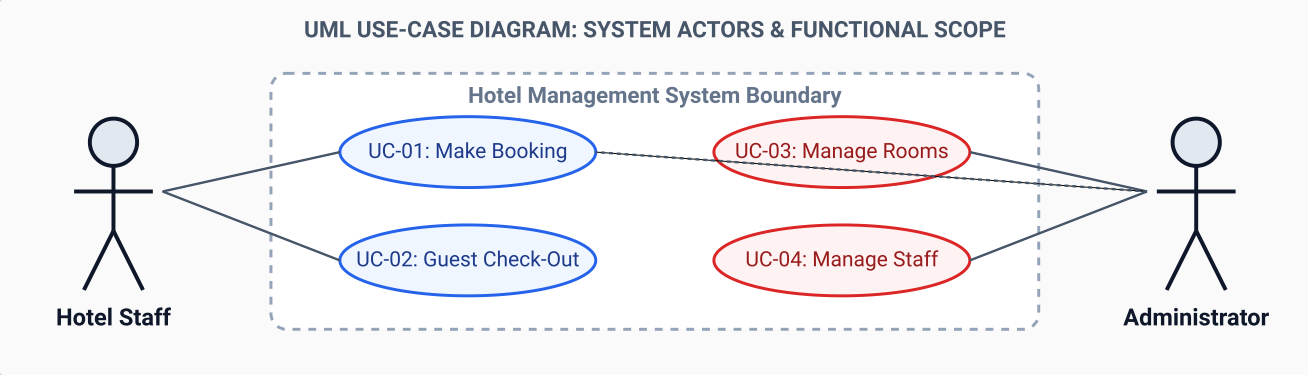
3. Non-Functional Requirements (5 Objective Criteria)

NFR ID	CATEGORY	SYSTEM GOAL	OBJECTIVE MEASURABLE VERIFICATION CRITERIA
NFR-001	Performance	Real-Time Search Response	Filtered UI list updates must execute and render within ≤ 200 ms upon key release in the search bar.
NFR-002	Security	Role-Based Access Control	Administrative controls (Room edits/deletions, Staff management) must be completely hidden/disabled for Staff logins.

NFR ID	CATEGORY	SYSTEM GOAL	OBJECTIVE MEASURABLE VERIFICATION CRITERIA
NFR-003	Data Integrity	Strict Input Field Validation	Phone numbers must strictly contain digits (`isdigit()`) and emails must contain `@` and `.` before saving bookings.
NFR-004	Usability	Active Session Monitoring	The status bar timer must continuously display elapsed session duration, updating precisely at 1.0 second intervals.
NFR-005	Availability	Fault-Tolerant State Recovery	If local configuration files are missing at launch, the system must auto-generate default fallback records without crashing.

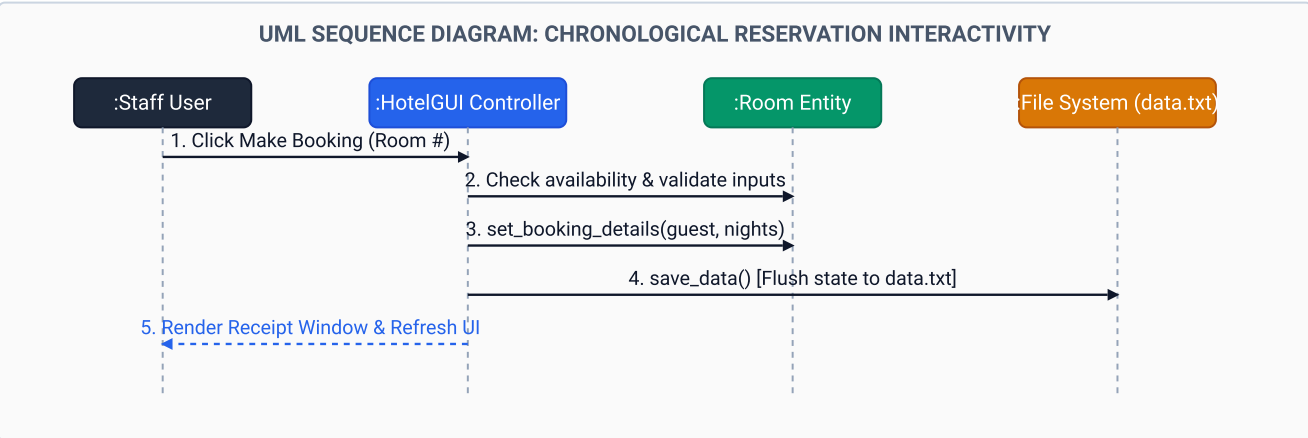
TASK 2: SYSTEM DESIGN REPORT (UML DIAGRAMS)

1. Use-Case Diagram & Tabular Specifications

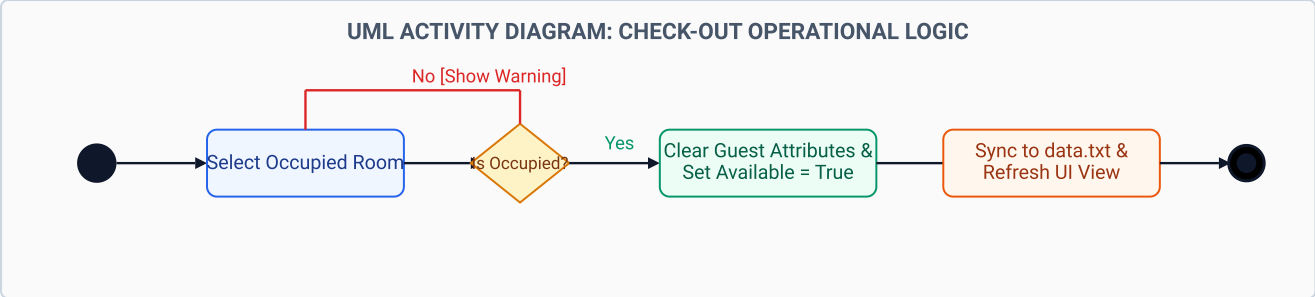


USE CASE FIELD	UC-01: MAKE ROOM RESERVATION (FR-006)	UC-04: MANAGE STAFF ACCOUNTS (FR-002)
Primary Actors	Hotel Staff, Administrator	Administrator Exclusive
Preconditions	Target room exists and holds 'Available' status.	Admin session is active with valid privileges.
Main Flow	1. Select room & trigger Booking. 2. Enter Guest Name, Phone, Email & Stay Duration. 3. System validates inputs, updates room to 'Occupied', persists to `data.txt`, and pops up receipt GUI.	1. Select Action (Add / Delete). 2. Input credentials (Username, Password, Role). 3. System validates constraints, updates local accounts dictionary, and flushes to `staff.txt`.
Alternative Flow	If inputs fail format checks (non-digits in phone), throw alert dialog and re-prompt user.	If deleting active session user or duplicate username, display warning alert and halt action.

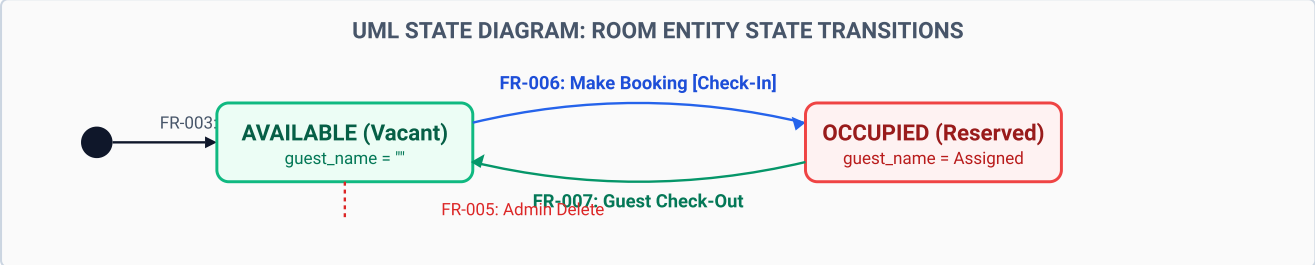
2. Sequence Diagram (Key Scenario: Room Booking Workflow)



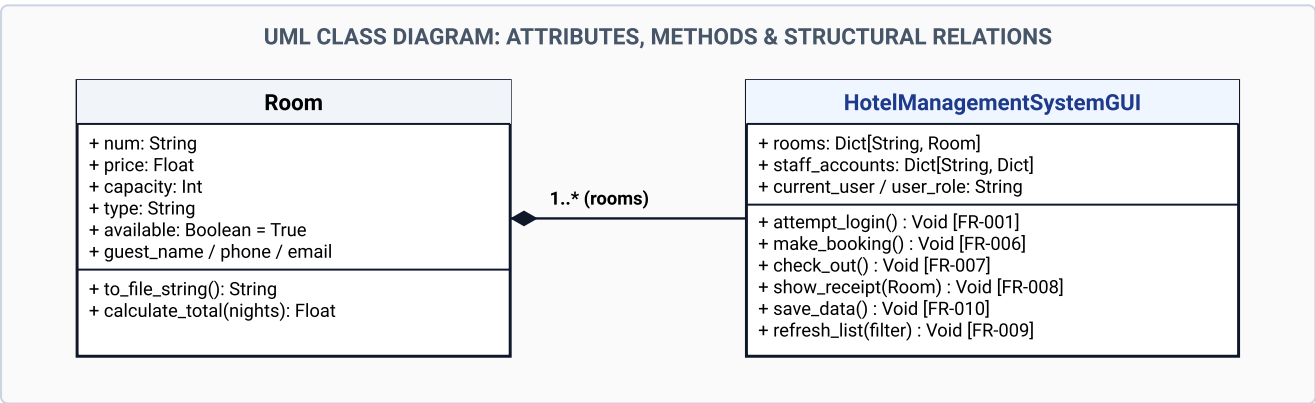
3. Activity Diagram (Check-Out Process Workflow)



4. State Diagram (Room Lifecycle Entity States)



5. Class Diagram (System Domain Model & Controller Architecture)



TASK 3: FUNCTIONAL PROTOTYPE IMPLEMENTATION

1. Requirement Prioritization Matrix (Top 5 Selected)

RANK	REQ ID	REQUIREMENT DESCRIPTION	STATUS	SELECTION & IMPLEMENTATION JUSTIFICATION
1	**FR-001**	User Authentication & Access Roles	**Selected (1)**	Primary portal layer enforcing operational system security and role distinction.
2	**FR-006**	Room Reservation Workflow	**Selected (2)**	Core domain feature executing primary business logic and guest record association.
3	**FR-007**	Guest Departure & Check-Out	**Selected (3)**	Critical lifecycle operation releasing inventory back to available state.
4	**FR-008**	Financial Receipt Generation	**Selected (4)**	Required financial audit output providing itemized billing data export.
5	**FR-010**	Local Data Persistence (File I/O)	**Selected (5)**	Underlying storage engine guaranteeing state persistence across restarts.

2. Code Traceability & Implementation Proof (Python 3 / Tkinter)

Below are verbatim, fully functional code segments extracted directly from the prototype script (`hotel.pyw`), confirming exact implementation compliance:

```
# ============================================================================
# TRACEABILITY PROOF 1: FR-001 (Authentication) & FR-010 (File Persistence)
# ============================================================================
def attempt_login(self, event=None):
    user = self.u_entry.get().strip().lower()
    pwd = self.p_entry.get().strip()

    if user in self.staff_accounts and self.staff_accounts[user]["pass"] == pwd:
        self.current_user = user
        self.user_role = self.staff_accounts[user]["role"]
        self.login_win.destroy()
        self.root.deiconify()
        self.build_gui()
        self.refresh_list()
    else:
        messagebox.showerror("Access Denied", "Invalid username or password.")

# ============================================================================
# TRACEABILITY PROOF 2: FR-006 (Booking), FR-008 (Receipt) & NFR-003 (Validation)
# ============================================================================
def make_booking(self):
    room = self._get_target_room("Booking", "Enter Room Number to reserve:")
    if not room or not room.available:
        return

    # Input Capture & Validation Checks (NFR-003)
    g_name = simpledialog.askstring("Guest Name", "Enter Guest Full Name:")
    g_phone = simpledialog.askstring("Phone", "Enter Phone Number (Digits only):")
    g_email = simpledialog.askstring("Email", "Enter Email Address (@ required):")

    if not (g_phone and g_phone.isdigit()) or not (g_email and "@" in g_email and "." in g_email):
        messagebox.showerror("Validation Error", "Invalid Phone Number or Email format!")
        return

    nights = simpledialog.askinteger("Stay Duration", "Enter number of nights:", minvalue=1)

    # Execute Booking State Update
    room.guest_name, room.guest_phone, room.guest_email = g_name, g_phone, g_email
    room.nights, room.available = nights, False

    self.save_data()      # Persists updated state to data.txt (FR-010)
    self.refresh_list()   # Updates Treeview GUI dynamically (FR-009)
    self.show_receipt(room) # Spawns printable receipt window (FR-008)
```